Ders 3 Çalışma Özeti

Ders 3 Çalışma Özeti

Genel Konular

- Factory Method kalıbı: amaç ve örnek (ekrandan, 00:10:00)
  - Amaç: bir nesne oluşturmak için öyle bir arayüz sunmak ki, oluşturulacak nesnenin sınıfına bu arayüzü sağlayan sınıfın alt sınıfları karar verebilsin.
  - Örnek: farklı belge türleri (metin, resim, karışık; Word ve Excel benzeri) ile çalışabilecek uygulamalar geliştirmeyi kolaylaştıran bir çerçeve program (framework). Temel sınıflar Belge (Document) ve Uygulama (Application); kullanıcı programcı bu iki sınıftan kalıtımla yeni sınıflar türetir.
  - Sorun: "yeni", "aç" gibi komutlarda çerçeve program bir belgeyi nasıl oluşturacak? Uygulama, kullanıcının türeteceği belge sınıflarını önceden bilemez.
- Factory Method örnek çözümü (ekrandan, 00:20:00)
  - Soyut Application sınıfında soyut CreateDocument() ile somut NewDocument() ve OpenDocument() metotları; soyut Document sınıfında soyut Open() ve Close() ile somut Save() ve Revert() metotları var. Application ile Document arasında "docs" adlı parça-bütün ilişkisi kurulur.
  - NewDocument() gövdesi: `Document* doc = CreateDocument(); docs.Add(doc); doc->Open();`. MyApplication alt sınıfı CreateDocument()'i `return new MyDocument` diye gerçekler.
  - Ana fikir: somut metotlar, akışın en gerekli yerlerinde soyut metotları çağırır; böylece kullanıcı programcının yalnız farklılaşması gereken adımları kodlayacağı bağlantı noktaları oluşur.
  - Diğer seçenek: Java'da generic sınıflar (`class Document<DocType>`, C++'taki template sınıflar gibi). Zayıf yönü dile özel olmasıdır; taşınabilirliği azaltır.
- Factory Method kalıp yapısı (ekrandan, 00:21:40)
  - Creator (soyut): soyut FactoryMethod() ve onu kullanan AnOperation() (`product = FactoryMethod()`).
  - ConcreteCreator: FactoryMethod()'u `return new ConcreteProduct` olarak gerçekler. Product soyut, ConcreteProduct onun alt sınıfı.
  - Creator'dan Product'a sahiplik ya da parça-bütün ilişkisi kalıp için zorunlu değildir; uygulama mantığı gerektiriyorsa eklenir. Ters yönde ilişki anlamsızdır: ürün kim tarafından oluşturulduğunu bilmek zorunda olmamalıdır.
- Factory Method'un uygulanabileceği anlar (ekrandan, 00:26:20)
  - Yeni nesneler oluşturması gereken bir sınıfın, oluşturacağı nesnenin türünü bilemediği (ya da bilmesinin istenmediği) yerlerde.
  - Bir sınıfın kendi üzerindeki bir sorumluluğu çeşitli yardımcı alt sınıflarına ileteceği ve hangi tür yardımcı alt sınıfın oluşturulacağı bilgisinin yerelleştirilmesi gerektiği yerlerde.
- Çizim uygulaması örneği (ekrandan, 00:30:00, 00:31:10)
  - Şekiller fare ile değiştirilecek. Soyut Figure sınıfında CreateManipulator(); LineFigure ve TextFigure alt sınıfları sırasıyla LineManipulator ve TextManipulator oluşturur. Soyut Manipulator'ün DownClick(), Drag(), UpClick() metotları vardır; Client hem Figure hem Manipulator ile ilişkilidir.
  - Örneğin metin şekli sürüklenince taşınır; Bezier eğrisi (spline) gibi bir çizgide özel noktalar sürüklenince eğim değişir.
  - Burada Creator figürler, Product manipülatörlerdir; asıl iş oluşturma değil kullanmadır. Alt sınıflarda ters yönde kullanma ilişkisi de vardır, kalıp bu okla genişletilebilir.

  - Bazı şekil türleri özel bir değiştiriciye gerek duymuyorsa Figure sınıfı varsayılan bir değiştirici örneği döndürebilir; bu durumda her şekil için ayrı değiştirici sınıfı yazmaya gerek kalmaz.
- Factory Method örnek gerçeklemesi: e-ticaret sitesi (ekrandan, 00:35:50, 00:37:40, 00:38:50, 00:40:00)
  - Problem: site farklı tür ürünler (elektronik, kitap, giyim, kırtasiye) satıyor. Sipariş işleme adımları belli bir sırada yürütülüyor, ama adımlardaki iş mantığı ürün türüne göre çok farklı (garanti belgesi, kargo, depo, tedarikçi). Site çalışanına sipariş vermede kullanabileceği bir araç sağlanacak. Head First Design Patterns kitabındaki bir örnekten esinlenilmiştir.
  - Çözüm: Product rolünde soyut Order (prepare, orderFromSupplier, pack, sendToCustomer soyut metotları) ve somut Electronics, Books sınıfları. Creator rolünde soyut Store (soyut, protected `createOrder(String uniqueID): Order` ve somut `makeOrder(String uniqueID)`) ile somut ElectronicsStore ve BookStore sınıfları.
  - makeOrder(), createOrder() ile siparişi oluşturur, sonra prepare, orderFromSupplier, pack ve sendToCustomer adımlarını sırayla çağırır. ElectronicsStore.createOrder() `new Electronics(uniqueID)` döndürür.
  - Bu örnekte alternatif çözümler karşılaştırılmadı, yalnız kalıbın uygulanışı gösterildi.
- Builder kalıbı: amaç ve GoF örneği (00:52:00 civarı, ekrandan 01:00:00)
  - Amaç: karmaşık bir nesnenin oluşturulmasını gösteriminden ayırmak; böylece aynı oluşturma işleminden farklı gösterimler elde edebilmek. Karmaşık varlık, alt parçalarının farklı biçimlerde oluşturulup birleştirilmesiyle elde edilir.
  - GoF örneği: RTF (Rich Text Format) belge okuyucu. Belge metin dışında grafik, ses, video gibi bileşenler içerebilir. Okunan belge farklı metin biçimlerine, hatta düzenlenebilir bir GUI bileşenine çevrilebilmeli; yeni biçimler ileride kolayca eklenebilmeli.
  - Çözüm: RTFReader'ın ParseRTF() metodu token'ları sırayla alır ve türüne göre (CHAR, FONT, PARA) soyut TextConverter'ın ConvertCharacter(char), ConvertFontChange(Font), ConvertParagraph() metotlarını çağırır. Somut alt sınıflar ASCIIConverter, TeXConverter, TextWidgetConverter; ürünleri sırasıyla ASCIIText, TeXText, TextWidget (GetASCIIText(), GetTeXText(), GetTextWidget()).
  - Çözümün eleştirileri (slayttan): çok biçimlilik tam kullanılamadı, switch-case hâlâ duruyor; alt sınıflar üst sınıfın tüm metotlarını gerçeklemek zorunda olduğundan anlamı olmayan metotlar boş gövdeli kalıyor (ASCIIConverter'da font değişimi gibi).
- Builder kalıp yapısı ve bileşenleri (ekrandan, 01:10:00, 01:10:20)
  - Director (Construct(); `for all objects in structure { builder->BuildPart() }`), soyut Builder (BuildPart()), ConcreteBuilder (BuildPart(), GetResult()) ve Product.
  - Builder: ürün nesnesinin parçalarını oluşturmak için bir arayüz sunar. Product: kendisini oluşturan parçaları tanımlayan sınıfları ve parçaları bir bütün haline getirmek için gerekli arayüzleri içerir. Director: ürünü Builder arayüzünü kullanarak oluşturur. ConcreteBuilder: parçaları oluşturur ve birleştirir, oluşturduğu gösterimi tanımlar ve izler, ürünü geri vermek için bir getter sunar.
  - Builder'ın bütün metotları soyutsa Java'daki interface olarak da tanımlanabilir.
- Builder bileşenlerinin etkileşimi (ekrandan, 01:11:50)
  - Client önce bir ConcreteBuilder, sonra bu builder ile bir Director oluşturur; Director'ın Construct() metodunu çağırır. Director sırayla BuildPartA(), BuildPartB(), BuildPartC() çağırır. Son olarak Client GetResult() ile ürünü ConcreteBuilder'dan alır.
- Builder'ın uygulanabileceği anlar ve sonuçları (ekrandan, 01:16:10)
  - Karmaşık bir nesneyi oluşturma algoritmasının, nesneyi oluşturan parçalardan ve bu parçaların nasıl birleştirileceğinden bağımsız olması gerektiğinde.
  - Bir nesneyi oluşturma sürecinin, nesnenin farklı gösterimlerine izin vermesi gerektiğinde.
  - Sonuç: ürünün iç yapısı (hocanın düzeltmesiyle daha çok gösterimi) yeni bir ConcreteBuilder yazarak en az yan etkiyle değiştirilebilir. Factory kalıplarına göre ürün oluşturma süreci üzerinde daha fazla denetim vardır: Director, Builder'a hangi işlemi hangi sırada yapacağını söyler.
- Builder örnek gerçeklemesi: otomobil bayisi (ekrandan, 01:20:00, 01:30:00, 01:36:00, 01:40:00, 01:43:20, 01:45:40, 01:48:30)
  - Problem: farklı marka ve modellerin satıldığı bir bayide, markalara özel adlandırma ifadelerine rağmen ortak bir araç modellemesi yapılacak. Araç; kasa, yakıt türü, motor hacmi ve gücü, vites gibi bileşenlerin bir araya getirilmesiyle karmaşık bir süreçle

oluşur.
  - product paketi: Car (brand, model: String), Transmission (gearCount: int {final}, isAutomatic: boolean {final}), Engine (volume, power: int), «enum» Chassis (SEDAN, HATCHBACK, STATIONWAGON, SUV), «enum» FuelType (PETROLEUM, DIESEL, LPG, ELECTRIC).
  - example paketi: CarDirector (builder: CarBuilder; construct(elements[]: String), report()), «abstract» CarBuilder (product: Car; `# getProduct(): Car`, `+ getResult(): Car`, soyut buildChassis(), buildEngine(token), buildTransmission(token)), somut RenaultBuilder ve HondaBuilder. Hoca sınıf adlarına kalıptaki rol adlarını (Director, Builder) bilerek ekledi.
  - CarDirector.construct(): önce `builder.buildChassis()` çağrılır; sonra her eleman sekme (`\t`) karakterinden ikiye bölünür, "Engine" ise buildEngine(info), "Gear" ise buildTransmission(info) çağrılır. report(), `builder.getResult()` sonucunu yazdırır.
  - main: `new CarDirector(new HondaBuilder("Civic Sedan"))`, elemanlar `{"Engine\t125PS", "Engine\t1.6L", "Gear\t6AT"}`. Çıktı: `Engine: Petroleum 1600cc 125HP` ve `Transmission: 6 gears automatic transmission`.
  - HondaBuilder.buildEngine(): motor yoksa varsayılan `new Engine(FuelType.PETROLEUM, 1150, 60)`; token büyük harfe çevrilir; "ECO" içeriyorsa yakıt LPG; "PS" içeriyorsa öncesindeki sayı güç olarak; "L" içeriyorsa sayı motor hacmi olarak işlenir.
  - getResult(), ürünü doğrudan değil, alanları kopyalanmış bir klonunu döndürür (product null ise null döner). getProduct() ise alt sınıfların ürüne erişmesi için protected'dır.
  - Yeni bir marka satılacağında yalnız CarBuilder'ın yeni bir kardeş alt sınıfı (örneğin FordBuilder) eklenir; CarDirector ve CarBuilder'a dokunulmaz.

Hocanın Özellikle Vurguladığı Kısımlar

- Abstract Factory ile Factory Method'u karıştırmayın
  - Abstract Factory'de ürün aileleri ve bu ailelerin üyeleri vardır (örneğin pencere yöneticisi aileleri ve onların radio button, check button, input box üyeleri). Factory Method'da aile hiyerarşisi yoktur. En temel ayrım budur.
- Builder'ı factory kalıplarından ayıran şey denetimdir
  - Factory kalıplarında istemci "bana şu üründen ver" der ve alır. Builder'da Director, parçaların ürüne katılması üzerinde daha fazla denetime sahiptir; isterse bazı parçaları göndermeyerek bir yarı mamul de oluşturabilir.
- Kalıplar harfiyen uygulanacak çözümler değildir
  - Problemin ayrıntılarına göre kalıbın önerdiği ilişkilerin üstüne ek ilişkiler (sahiplik, parça-bütün) kurulabilir. Ama kalıbın önerilerinin çok azı alınıp çoğu dışarıda bırakılırsa artık o kalıp uygulanmamış olur; bu her zaman kötü değildir, avantaj ve dezavantajlara göre karar verilir.
- Dile özel yetenekler yerine kalıplar
  - Generic sınıflar gibi dile özel çözümler taşınabilirliği azaltır. Nesneye yönelik çözümler öneren kalıplar üzerinden gitmek kodun taşınabilirliğini iyileştirir ve bakım masrafını düşürür.
- Switch-case'i olabildiğince az yerde tutun
  - Builder yapısında switch-case gözükmese de ConcreteBuilder.buildPart içinde kendini gösterecektir, çünkü yapıdaki nesneler bir şekilde parametre olarak aktarılıp "parça şu ise şunu yap" denmek zorunda kalınacaktır. Birden fazla somut Builder içinde switch-case kullanmaktansa Director.construct içinde tek switch-case kullanmak daha iyidir (ekrandan, 01:10:00).
- GoF şemalarını güncel UML'ye göre çizin
  - GoF kitabı UML standartlaşmadan yazıldığı için kalıtım okunun ucu üst sınıftan uzakta çizilmiştir; bu çizim karga ayağı gösterimiyle karışabilir. Sınavlarda güncel UML standardına göre çizilmelidir.
- Sıralama şemasında dönüş oklarını çizin
  - GoF sıralama şemalarında dönüş okları pek çizilmez; çizilmezse bir metodun diğerinin bitiminden sonra mı yoksa içinde mi yürütüldüğü net anlaşılmaz.
- Yazılım sürece uymalı
  - Süreç hazır yazılıma uydurulmamalıdır; yazılım iş sürecine uymalıdır.
- En esnek tasarım: yeni ihtiyaç için yalnız yeni sınıf eklemek

  - Bir şey eklemek için yalnız bir sınıf eklemek ve başka yere dokunmamak en rahat bakım kolaylığıdır. Başka yerlere dokunmak gerekiyorsa tasarım yeterince esnek değildir.

Kısa Tekrar Notları

- Factory Method: nesne oluşturma arayüzünü üst sınıf sunar, hangi sınıftan nesne oluşturulacağına alt sınıflar karar verir.
- Roller: Creator (soyut FactoryMethod + onu kullanan AnOperation), ConcreteCreator, Product, ConcreteProduct.
- Belge çerçevesi: Application.NewDocument() → CreateDocument() (soyut) → docs.Add → doc.Open(). MyApplication, MyDocument döndürür.
- Çizim uygulaması: Figure.CreateManipulator(); gerek yoksa varsayılan bir manipülatör döndürülür.
- E-ticaret: Store.makeOrder() sabit adımları yürütür, Store.createOrder() soyut; ElectronicsStore ve BookStore somut sipariş nesnesini oluşturur.
- Abstract Factory = ürün aileleri; Factory Method = aile yok.
- Builder: karmaşık nesnenin oluşturulmasını gösteriminden ayırır. Roller: Director, Builder, ConcreteBuilder, Product.
- RTF okuyucu: RTFReader (Director), TextConverter (Builder), ASCII/TeX/TextWidget çeviriciler (ConcreteBuilder). Eleştiri: switch-case kalıyor, boş gövdeli metotlar.
- Etkileşim sırası: new ConcreteBuilder → new Director(builder) → Construct() → BuildPartA/B/C → GetResult().
- Builder, factory kalıplarına göre oluşturma süreci üzerinde daha fazla denetim verir; yeni gösterim eklemek kolay, iç yapıyı değiştirmek daha zordur.
- Otomobil bayisi: yeni marka = yeni CarBuilder alt sınıfı; getResult() klon döndürür.

Detaylı Açıklamalar

- Factory Method'da soyut ve somut metotların iş bölümü
  - Çerçeve programın amacı, belge türünden bağımsız olarak yapılabilecek ortak işleri (yeni belge oluşturma komutunu başlatma, açma diyalogunu başlatma, açık belge listesini yönetme, kaydetme işlemini başlatma, geri alma) kullanıcı programcının üzerinden almaktır. Bu işler somut metotlarda ayrıntılı olarak kodlanır.
  - Belge türüne göre değişen işler soyut metot olarak bırakılır ve somut alt sınıflarda gövdeleri doldurulur. Somut NewDocument() içinden soyut CreateDocument() çağrıldığında, çok biçimlilik sayesinde MyApplication'ın gerçeklemesi çalışır ve MyDocument nesnesi döner; Document beklenen yerde alt sınıf nesnesi kullanılabilir.
  - Şemadaki "docs" adlı parça-bütün ilişkisi, büyük olasılıkla bağlı liste gibi bir veri yapısıyla gerçeklenir; dil kitaplıklarındaki add, search, remove metotları kullanılır. Hoca Revert() için geri alma/yineleme işlemlerinin ileride görülecek Memento kalıbıyla kodlanabileceğini de belirtti.
  - Ürün kendisini oluşturanı bilmek zorundaysa, Creator kendi referansını (this) factory method içinde ürüne parametre olarak verebilir; ama somut alt sınıfın bunu doğru vereceğinin garantisi yoktur.
- E-ticaret örneğinin kodu (ekrandan, 00:38:50, 00:40:00)
  - Order soyut sınıfının dört metodu da soyuttur, çünkü ne yapılacağı belli ama nasıl yapılacağı ürün türüne göre çok farklıdır. Electronics, kurucusunda `super()` çağırıp barcode alanını atar ve dört metodu elektronik malzemeye özgü olarak gerçekler; Books benzer biçimde kodlanır.
  - Store'da ortak işler somut makeOrder() içindedir: siparişi oluşturduktan sonra hazırlama, tedarikçiden temin, paketleme ve müşteriye gönderme. Farklılaşan tek adım oluşturma olduğu için createOrder() soyut bırakılır.

```
public abstract class Store {
    protected abstract Order createOrder( String uniqueID );
    public void makeOrder( String uniqueID ) {
        Order newOrder = createOrder( uniqueID );
        newOrder.prepare( );
        newOrder.orderFromSupplier( );
```

```
            newOrder.pack( );
            newOrder.sendToCustomer( );
        }
    }
    public class ElectronicsStore extends Store {
        protected Order createOrder( String uniqueID ) {
            Electronics newElectronicsOrder = new Electronics(uniqueID);
            //Elektronik malzeme siparişi için yapılacak işlemlerin tanımlanması
            return newElectronicsOrder;
        }
    }
```

  - Ders sonrası not: problem slaytındaki kullanım örneğinde (ekrandan, 00:35:50) istemci `store.createOrder("5524345678")` çağırıyor. createOrder() protected'dır ve yalnız sipariş nesnesini oluşturur; adımları yürüten makeOrder() olduğundan istemcinin büyük olasılıkla `store.makeOrder(...)` çağırması amaçlanmıştır. Hoca bu farkı derste belirtmedi.
- Builder'da switch-case'in yeri ve RTF çözümünün eleştirisi
  - RTF örneğinde dördüncü bir biçim (örneğin PDF) eklemek yalnız yeni bir çevirici alt sınıfı gerektirir. Asıl sorun ürüne yeni bir alt bileşen türü (örneğin gömülü video) eklenmesidir: soyut TextConverter'a ConvertVideo() eklenir eklenmez eski bütün alt sınıflar, kendilerinde anlamı olmasa bile bu metodu boş gövdeyle gerçeklemek zorunda kalır; RTFReader'daki switch'e de yeni bir case eklenir.
  - Hoca bunu bir kod kusuru olarak niteledi: hata değil, ama esnekliği azaltan ve nesneye yönelimin önerdiği kullanım biçimlerinin dışında kalan bir kullanım. Yine de çözümsüz kalmaktan ya da daha karmaşık bir çözümden iyidir.
  - Switch-case somut Builder'lara da taşınabilir (Director token'ı olduğu gibi verir, builder türüne bakar), ama somut builder sayısı fazla, Director bir tane olduğundan switch daha çok yerde üretilmiş olur. Hoca bu yüzden Director'da yapılmasını tercih etti.
- Otomobil bayisi örneğinde Director ile Builder arasındaki iş bölümü
  - Marka adlandırmaları karmaşıktır: örneğin Honda'da model adında CR-V geçiyorsa SUV'dir; bir başka markada ürün kodunun basamak sayısı kasa türünü belirleyebilir. Bu bilgi değişen şey olduğu için ona en çok vakıf olan somut marka Builder'ına konur.
  - CarDirector ise yalnız bileşen türlerini (motor, vites, kasa) bilir; markayla ilgili hiçbir şey içermez. Token'ın türünü ("Engine", "Gear") Director ayırt eder, değerin markaya göre yorumlanması Builder'da yapılır. Kasa bilgisi model adından (örneğin "back" geçiyorsa hatchback, aksi halde varsayılan sedan) çıkarılır.
  - Hoca bir an örnekte tüm token'ların Builder'a gönderildiğini (bir tür denetimin tersine çevrilmesi, inversion of control) söyledi, kodu açınca bunu geri aldı: ayrıntılı token kontrolü Director'da yapılmaktadır.
  - getResult() ürünün klonunu döndürür. Builder'ın product alanı doğrudan döndürülseydi, setter olmasa bile referans korunduğu için dışarıdan nesne üzerinde değişiklik yapılıp ürün yanlış çalışacak biçimde bozulabilirdi. Klonda yapılan değişiklik asıl ürünü etkilemez.
  - Tehdit: araçlara yeni bir alt parça eklenirse (örneğin otonom sürüşle ilgili bir bileşen) Builder'a yeni metot ve Director'a yeni dal gerekir. Hocaya göre araçların bileşenleri büyük ölçüde belli olduğu için bu olasılık düşüktür.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.